



HGANN-Mal: A Hypergraph Attention Neural Network Approach for Android Malware Detection

Mohammad Reza Norouzian^(✉) and Claudia Eckert

Technical University of Munich, Munich, Germany

mohammad.norouzian@tum.de, claudia.eckert@in.tum.de

Abstract. Research on Android malware has progressed rapidly, yet the task of distinguishing malicious from benign applications continues to test the limits of automated analysis. Earlier work, dominated by static signature matching, frequently struggles when novel or obfuscated samples appear. Contemporary graph-based pipelines alleviate some of these shortcomings by modelling control and data dependencies, but their reliance on pairwise relations often blurs higher-order interactions that experienced adversaries nurture when crafting evasive variants. These observations motivate a return to first principles: we require representations that faithfully encode behaviours without incurring prohibitive overhead. In this study we revisit the problem through the lens of hypergraph representation learning. Treating an application as a hypergraph allows one to encode joint behaviours—such as the co-invocation of critical API calls inside a single execution context—that cannot be decomposed into simple edges without information loss. Building on this representation, we introduce HGANN-Mal, a Hypergraph Attention Neural Network that adaptively emphasises semantically salient hyperedges while softening the influence of spurious ones. The model derives its signals from static analysis to extract structural and semantic features. Importantly, on the Drebin dataset, HGANN-Mal achieves a Macro-F1 score of 97.8% and an accuracy of 98.3% in binary malware detection, significantly outperforming graph-based and static hypergraph methods. Our findings validate that the proposed attention-based hypergraph model provides a more exhaustive and precise solution for detecting sophisticated Android malware.

1 Introduction

Android's open distribution model and broad hardware ecosystem have propelled the platform to a market share that regularly exceeds with over 2.5 billion active devices [17]. That ubiquity, however, does not come without cost: security analysts continue to report hundreds of thousands of suspicious packages each year. While the absolute numbers fluctuate with collection methodology, the consensus view is that the threat landscape is both quantitatively significant and qualitatively diverse [4].

Traditional defences have long relied on signature-based [28], where an application’s code sections are compared against a repository of known malicious fragments. Such detectors remain valuable thanks to their interpretability and low computational overhead. At the same time, they provide little protection against so-called zero-day malware or against families that rotate byte sequences through polymorphic engines [43]. In response, researchers have shifted their attention towards machine learning-based systems capable of extracting higher-level abstractions directly from raw artifacts.

Among the techniques now popular, graph neural networks (GNNs) occupy a key role. Function call graphs (FCGs), permission-API dependency graphs, and inter-component communication graphs have all served as substrates for deep learning architectures [8]. Although the empirical results are impressive, a recurring critique concerns their pairwise bias: edges express binary relations, yet many security-relevant behaviours emerge only when three or more components interact [5, 16]. Hypergraphs, with their capacity to join arbitrarily large node sets, offer a principled remedy.

Hypergraphs extend standard graphs by allowing a hyperedge to connect an arbitrary number of nodes [11]. The structure is therefore capable of recording composite behaviours without having to enumerate every pairwise interaction. Recent studies in computer vision and recommendation systems suggest that hypergraph neural networks (HGNNs) [11, 14] can leverage this richer topology to learn more expressive embeddings. In the Android domain, preliminary evidence is beginning to surface [45], yet the question of how to weigh the relative contribution of distinct hyperedges remains unsettled.

The construction of hypergraphs for Android applications can follow various strategies. Feng et al. [45] capture structural semantics by creating hyperedges from groups of functions that share the same sensitive permission requests or are located within k -order call graph neighborhoods. Wu et al. [41] explored a dual-view approach that constructs hypergraphs from both structural and semantic perspectives, capturing both the call relationships and the functional similarities between components.

Experimental evaluations on benchmark datasets such as Drebin [5] and CICMalDroid [26, 27] have demonstrated that hypergraph-based approaches consistently outperform traditional graph-based methods in Android malware detection tasks. For instance, HGNN+ [14] achieved classification accuracy improvements of 3–5% over state-of-the-art GNN models by better capturing the complex relationships between application components. These results underscore the importance of modeling higher-order relationships in Android applications for effective malware detection.

Despite these advances, existing hypergraph-based methods employ static, pre-defined hyperedge constructions that assign equal importance to all relationships. This uniform weighting fails to recognize that certain functional relationships are more indicative of malicious behavior than others. For example, interactions involving sensitive API calls or dangerous permission requests should likely receive greater emphasis than routine utility function calls [29]. The inability to

adaptively weight these relationships limits the discriminative power of current hypergraph representations.

To address these limitations, we propose a novel approach that integrates attention mechanisms [6] into hypergraph neural networks for Android malware detection. HGANN-Mal, extends beyond static hypergraph structures by dynamically learning the importance of different hyperedges through an attention mechanism. This approach enables the model to focus on the most discriminative functional relationships while downweighting less relevant connections. In summary, we make the following primary contributions:

- We formulate Android malware detection as a hypergraph classification problem, leveraging the expressive power of hypergraphs to model complex, higher-order relationships between functions in Android applications.
- We introduce a novel hypergraph attention mechanism that dynamically learns to weight hyperedges based on their relevance to malware detection.
- We demonstrate that adaptive weighting of hyperedges through attention improves discrimination between benign and malicious.

Our approach builds upon recent advances in hypergraph neural networks [6, 14] and attention mechanisms in graph learning [38, 39]. Attention mechanisms have proven effective in graph neural networks by allowing models to focus on important neighbors [39], but their application to hypergraph structures remains largely unexplored, particularly in the context of malware detection.

2 Related Work

Android malware detection has evolved significantly from traditional signature-based approaches to sophisticated machine learning and graph-based methods. Traditional malware detection techniques for Android platforms have primarily relied on static, dynamic, and hybrid analysis methods. Static analysis techniques examine application code without execution, focusing on features such as permissions, API calls, and code structure [33, 40]. Feng et al. [12] demonstrated the effectiveness of combining static taint analysis with program representation techniques, achieving notable accuracy in detecting malicious applications. Dynamic analysis methods monitor application behavior during runtime, e.g., capturing system calls with frequency vectors and co-occurrence matrices [42]. Hybrid analysis techniques combine static and dynamic approaches to leverage the strengths of both methodologies [31]. However, traditional approaches face significant limitations when confronting sophisticated evasion techniques such as code obfuscation, polymorphism, and metamorphism.

The limitations of traditional methods have driven researchers toward graph-based representations that capture structural relationships within Android applications. Function Call Graphs (FCGs) have emerged as a fundamental representation for modeling the invocation relationships between functions in Android applications [16]. Recent advances in Graph Neural Networks (GNNs) have revolutionized the application of FCGs to malware detection. Lu X. [24] proposed

SNDGCN, a robust Android malware detection model based on denoising Graph Convolutional Networks that processes FCGs while addressing noise in graph representations. Zheng J. [46] introduced MASKDROID, which employs masked graph representations to enhance the discriminative ability of malware detectors while maintaining robustness against adversarial attacks. Guo W. [18] developed MalHAPGNN, a Layered attention pooling graph neural network that enhances call graph representations through sophisticated attention mechanisms. Despite these advances, conventional graph representations remain fundamentally limited by their ability to model only pairwise relationships between entities, which cannot adequately capture complex interactions among multiple functions. [36].

Hypergraph structures offer a more expressive alternative to traditional graphs by allowing edges (hyperedges) to connect arbitrary numbers of vertices simultaneously. This capability naturally accommodates the complex, multi-entity relationships present in Android applications, where malicious behaviors frequently involve coordinated actions across multiple functions. Zhang et al. [45] pioneered the application of hypergraph neural networks to Android malware detection, proposing the first hypergraph-based approach for modeling higher-order relationships among functions. Their method constructs hypergraphs where functions sharing permissions or forming K-order neighborhoods constitute hyperedges, enabling the capture of complex semantic relationships that traditional graphs cannot represent. However, the static weights of hypergraph neural networks cannot capture the behavior of modern and complex new evolving of Android malware. In our HGANN-Mal approach addresses several critical gaps by employing hypergraph attention neural networks that extend beyond the limitations of traditional graph-based and hypergraph-based methods. Unlike prior Android HGNNs that use static, equally weighted k-hop/permission hyperedges [45], HGANN-Mal fuses topological, semantic, and slice-aware suspicious-API hyperedges and learns their importance via hypergraph attention and, to our knowledge, is the first to apply a learnable hypergraph-attention operator to Android malware detection, thereby amplifying malicious relations while suppressing noise.

3 Hypergraph Neural Networks

3.1 Preliminaries

A hypergraph is a more general topology than a graph that is capable of modeling multiple relationships between discrete data. Unlike traditional graphs where each edge connects exactly two vertices, a hyperedge in a hypergraph can connect multiple vertices simultaneously, thus enabling the modeling of higher-order relationships among vertices [1].

Formally, let $\mathcal{H} = \langle \mathcal{V}, \mathcal{E} \rangle$ be a hypergraph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ denotes the vertex set containing $|\mathcal{V}| = n$ vertices and $\mathcal{E} = \{e_1, e_2, \dots, e_m\}$ denotes the set of hyperedges, where each hyperedge $e \in \mathcal{E}$ is a subset of $\mathcal{V}$. In the context of Android malware detection, vertices can represent functions in an Android application, while hyperedges can represent higher-order relationships between

these functions, such as common call patterns or common suspicious API call combinations.

In a hypergraph, the vertex-hyperedge adjacency matrix is represented by $H \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{E}|}$. When a vertex v belongs to a hyperedge e , $H(v, e) = 1$, otherwise $H(v, e) = 0$. The weights of the hyperedges can be represented by a diagonal matrix $W \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{E}|}$ where $W(i, i) = w(e_i)$. The degree of vertices is computed by $d_{\mathcal{V}}(v) = \sum_{e \in \mathcal{E}} H(v, e) \cdot w(e)$, and the degree of hyperedges is computed as $d_{\mathcal{E}}(e) = \sum_{v \in \mathcal{V}} H(v, e)$. The matrix representation of the degree of vertices and hyperedges is denoted by $D_{\mathcal{V}}$ and $D_{\mathcal{E}}$, respectively. They are both diagonal matrices.

The Laplacian matrix of $\mathcal{H}$ is denoted by:

$$L = I - D_{\mathcal{V}}^{-1/2} H W D_{\mathcal{E}}^{-1} H^T D_{\mathcal{V}}^{-1/2} \quad (1)$$

where I is the identity matrix. This Laplacian matrix plays a crucial role in defining the spectral convolution operation on hypergraphs, which we will discuss in the following sections.

The ability of hypergraphs to model higher-order relationships makes them particularly suitable for Android malware detection, where complex interactions between functions, and API calls need to be captured for effective detection.

3.2 Hypergraph Neural Networks

HGNNs extend the concept of GNNs to hypergraphs, enabling the learning of node representations that capture higher-order relationships [11]. While traditional GNNs have shown great success in various domains, they are limited to modeling pairwise relationships. HGNNs overcome this limitation, where hyperedges can connect multiple vertices simultaneously [47].

The key challenge in designing HGNNs is to define appropriate operations for information propagation and aggregation over hypergraphs. In this section, we introduce several representative approaches for hypergraph neural networks, focusing on hypergraph convolution and hypergraph attention mechanisms, which form the foundation of our approach to Android malware detection.

Hypergraph Convolution. Hypergraph convolution defines a basic convolutional operator on a hypergraph, enabling efficient information propagation between vertices by fully exploiting the higher-order relationships encoded in hyperedges [6]. The primary challenge in defining a convolution operator on a hypergraph is to measure the transition probability between two vertices, which allows the features of each vertex to be propagated through the network.

To address this challenge, we make two key assumptions: (1) more propagation should occur between vertices connected by a common hyperedge, and (2) hyperedges with larger weights deserve more confidence in such propagation. Based on these assumptions, we define one step of hypergraph convolution as:

$$x_i^{(l+1)} = \sigma \left(\sum_{j=1}^N \sum_{e=1}^M H_{ie} H_{je} W_e x_j^{(l)} \Theta^{(l)} \right) \quad (2)$$

where $x_i^{(l)}$ is the embedding of the i -th vertex in the l -th layer, $\sigma(\cdot)$ is a non-linear activation function such as LeakyReLU [25] or ELU [9], and $\Theta^{(l)} \in \mathbb{R}^{F^{(l)} \times F^{(l+1)}}$ is the weight matrix between the l -th and $(l+1)$ -th layers.

It is worth noting that when hypergraph convolution is applied to a special case where each hyperedge connects exactly two vertices (i.e., a traditional graph), it reduces to the standard graph convolution operation. This property ensures that hypergraph convolution is a natural generalization of graph convolution to higher-order relationships.

Hypergraph Attention. While hypergraph convolution provides a powerful mechanism for information propagation on hypergraphs, it relies on a fixed, pre-defined structure for message passing. Hypergraph attention enhances this capability by introducing an attention mechanism that learns dynamic connections between vertices and hyperedges, allowing the model to focus on the most relevant parts of the hypergraph for a specific task [22, 39].

Hypergraph convolution already has a sort of innate attentional mechanism, however, this attentional mechanism is not learnable or trainable once the hypergraph structure (the incidence matrix H) is given.

The goal of hypergraph attention is to learn a dynamic incidence matrix, thereby creating a dynamic transition matrix that can better reveal the intrinsic relationships between vertices. Instead of treating each vertex as being either connected by a certain hyperedge or not, the attention module presents a probabilistic model, which assigns non-binary, real values to measure the degree of connectivity [6].

For a given vertex x_i and its associated hyperedge x_j , the attentional score is computed as:

$$\alpha_{ij} = \frac{\exp(\sigma(\text{sim}(x_i \Theta, x_j \Theta)))}{\sum_{k \in \mathcal{N}_i} \exp(\sigma(\text{sim}(x_i \Theta, x_k \Theta)))} \quad (3)$$

where $\sigma(\cdot)$ is a non-linear activation function, $\mathcal{N}_i$ is the neighborhood set of x_i , and $\text{sim}(\cdot)$ is a similarity function that computes the pairwise similarity between two vertices.

With the incidence matrix H enriched by an attention module, we can follow the same formulation as in hypergraph convolution to learn the intermediate embedding of vertices layer-by-layer:

$$X^{(l+1)} = \sigma(D_V^{-1/2} H_{\text{att}} W D_{\mathcal{E}}^{-1} H_{\text{att}}^T D_V^{-1/2} X^{(l)} \Theta^{(l)}) \quad (4)$$

where H_{att} is the attention-enriched incidence matrix. Note that hypergraph attention propagates gradients to H_{att} in addition to $X^{(l)}$ and $\Theta^{(l)}$, allowing the model to learn the optimal hypergraph structure for the task at hand.

In the context of Android malware detection, hypergraph attention allows our model to dynamically learn the importance of different relationships. For example, it can learn to assign higher attention weights to suspicious patterns of API calls requests that are indicative of malicious behavior.

4 Methodology

Our proposed Android malware detection framework leverages hypergraph attention neural networks. This section details the methodology, encompassing the initial data processing and static analysis, feature preparation, hypergraph construction, and the hypergraph-level classification process.

4.1 Data Acquisition and Static Analysis

Before delving into feature extraction and hypergraph construction, a robust initial phase of data acquisition and static analysis is critical.

Dataset Loader. The process (Stage 0) is responsible for systematically ingesting raw APK files from a designated root directory. The system walks through the directory structure, computing the SHA-256 hash of every APK file encountered. Subsequently, it queries and parses an external label list from the original dataset to tag each hash as either *benign* or the appropriate malware family. The output of this stage is a comprehensive catalogue, serves as the definitive manifest for the entire dataset.

APK Unpacker. Following the dataset loading, the APK Unpacker (Stage 1) takes the raw APK files as input. Tools such as ‘apktool’ and Python’s ‘zipfile’ module are employed to decompress and extract the ‘Android Manifest.xml’ file, which contains vital application metadata, and all ‘classes*.dex’ files, which encapsulate the application’s compiled Dalvik bytecode. The output of this stage comprises the raw manifest files and the raw DEX byte-streams, ready for deeper static analysis.

Static Analysis and API Tagging. The extracted manifests and DEX files are then processed through the MobiSF Static Analysis¹ & API Tagging component (Stage 2a), which leverages the MobiSF for comprehensive automated security analysis. The process involves several coordinated actions that work in tandem to identify and classify potentially suspicious API usage patterns [3, 35]. The analysis begins with an automated scan phase that MobiSF performs a comprehensive static analysis that includes manifest parsing, DEX disassembly, and security-focused feature extraction.

¹ <https://github.com/MobSF/Mobile-Security-Framework-MobSF>.

The next phase focuses on **permission risk assessment**, where each required permission identified in the previous step is cross-referenced. Only permissions whose **status** field is marked as "**dangerous**" are retained for further analysis. By focusing exclusively on dangerous permissions, the analysis concentrates on APIs that pose the highest potential security risk.

The framework then employs a **heuristic API classification** system to categorize the identified APIs into two primary categories based on their functional characteristics. APIs are classified as *access* APIs, such as those related to **Contacts**, **Location**, **Sms**, **Storage**, and similar sensitive data sources. Conversely, APIs are classified as *transmission* APIs, including those related to **Http**, **Socket**, **sendTextMessage**, and other network or communication functions. This heuristic approach, while not exhaustive, provides a practical and scalable method for identifying APIs that are commonly associated with data exfiltration patterns in malware.

To complement the MobiSF analysis and provide detailed call-site information, the system runs Androguard² in parallel for each APK. This parallel execution serves two primary purposes: first, it disassembles the DEX files to produce a comprehensive method-level Function-Call-Graph (FCG), denoted as $G = (V, E)$, where vertices V represent individual methods and directed edges E represent method invocations. Second, it generates a flat list of (**method_id**, **invoked_api**) pairs that capture the exact locations where specific APIs are called within the application's code structure.

The final step in this stage involves mapping MobiSF-flagged APIs back to their exact call sites using the Androguard-generated call-site information. This mapping process creates a comprehensive view of where potentially suspicious APIs are invoked within the application's execution flow, enabling precise localization of security-relevant code regions. Also, the stage produces a global FCG for the entire APK, derived from the Androguard analysis, which captures the structural relationships between methods and serves as the foundation for subsequent hypergraph construction phases. This dual-output approach ensures that both the security-relevant API usage patterns and the overall application structure are captured and available for downstream processing stages [23,37].

Backward-Slice Builder. Building upon the outputs of Stage 2a, the Backward-Slice Builder (Stage 2b) focuses on isolating security-critical code regions [34]. For every identified suspicious call site, this component performs two key actions:

- **Variable Retrieval:** It identifies variables that reach the suspicious call site. This involves analyzing data flow to determine which variables directly influence the behavior of the suspicious API call.
- **Slice Extraction:** It then walks the CFG backwards from the suspicious call site, retaining only basic blocks and statements that directly influence the identified variables. Each extracted slice is stored as a compact CFG file.

² <https://github.com/androguard/androguard>.

4.2 Feature Preparation

Effective representation of Android application characteristics is crucial for accurate malware detection. Building upon the outputs of the static analysis phase, we extract and process various features to comprehensively characterize the behavior of Android applications. These features serve as the input for the subsequent hypergraph construction and are designed to capture diverse aspects of an application's functionality and potential malicious intent. For each function $v \in V$ (where V is the set of vertices in the FCG), the following features are computed (Stage 3):

Opcode Histograms. To capture the low-level operational characteristics of each function, an opcode histogram is computed. This involves analyzing the Dalvik bytecode instructions within a function and generating a frequency distribution of opcodes. This histogram can be represented as a one-hot encoded vector or, for more comprehensive representations, using TF-IDF to reflect the importance of specific opcodes within a function relative to the entire corpus. Opcode patterns can reveal characteristic behaviors, including those associated with obfuscation or malicious payloads [19,30].

Structural Metrics. Beyond the raw opcode sequences, the structural position and influence of a function within the application's call graph provide valuable insights [10,13]. We calculate several structural metrics for each function, including:

- **In-degree:** The number of incoming edges to a function in the FCG.
- **Out-degree:** The number of outgoing edges from a function in the FCG, indicating how many other functions it calls.
- **PageRank:** A measure of a function's relative importance within the FCG, based on the number and quality of incoming links.
- **Betweenness Centrality:** A measure of a function's centrality in the FCG, quantifying the number of times it acts as a bridge along the shortest path between two other functions. Functions with high betweenness centrality often play critical roles in control flow.

These metrics are standardized to ensure consistent scaling across different applications and to prevent features with larger numerical ranges from dominating the learning process.

API-Related Flags and Counts. To directly incorporate insights from the suspicious API analysis (Stage 2a) and backward slicing (Stage 2b), we add specific API-related features for each method node:

- **has_access** and **has_transmit** flags: Binary flags indicating whether the method contains calls to sensitive data access APIs or sensitive data transmission APIs, respectively.

- Access/Transmit Call Counts in Slices: The number of access and transmit API calls that appear within the program slices associated with the method.

These flags and counts provide direct indicators of potentially malicious behavior at the method level.

Semantic Embedding (Code2vec). To capture the semantic meaning and contextual understanding of functions, we generate a 320-dimensional ‘code2vec’ embedding for each function [2]. This high-dimensional, dense embedding is chosen to capture rich contextual similarities between functions. While this approach carries a greater computational and memory footprint compared to handcrafted feature vectors, the trade-off provides a significantly more detailed understanding of the application’s true behavior. All computed features are then concatenated and z-score normalized to form the Node Feature Matrix X , where each row corresponds to a function (vertex) and each column represents a feature, resulting in a matrix of size $|V| \times F$, where F is the total number of features.

4.3 Construction of Hypergraph

To effectively model the intricate, higher-order relationships inherent in Android applications, we construct a hypergraph where functions serve as vertices and hyperedges represent complex interactions among multiple functions. Our hypergraph construction process involves defining three distinct types of hyperedges, followed by their integration into a unified hypergraph structure.

K-Hop Neighbour Hyperedge. Drawing inspiration from the work by Zhang et al. [45], we define k -order call neighbors as a fundamental feature for capturing the structural context of functions within an Android application (Stage 4A). For each node (function) in the FCG, its K -order callers and callees are collected to form a hyperedge. A typical value for K 3, which is sufficient to capture relevant local context. The output of this stage is the hyperedge list H_K .

Semantic-Embedding Hyperedge. Leveraging the semantic embeddings generated during feature preparation, we construct Semantic-Embedding Hyperedges (Stage 4B). This involves performing a k -nearest neighbor search on the function embeddings. Functions whose semantic embeddings are sufficiently similar are grouped into a hyperedge. A typical value for τ is approximately 0.80, and $k = 5$. This action captures functional similarities that might not be evident from call graphs alone, which can be particularly useful in detecting polymorphic malware or code reuse. The output of this stage is the hyperedge list H_{SEM} .

Suspicious-API Hyperedge. To highlight potentially malicious interactions, we introduce Slice-Aware Suspicious-API Hyperedges (Stage 4C). These hyperedges are formed by leveraging the sliced CFGs from Stage 2b and the suspicious-API tags. For each program slice, a hyperedge is created over all functions that

the slice contains. Each such hyperedge is assigned an initial weight w calculated as:

$$w = 1 + 0.2 \times (\# \text{ transmit}) + 0.1 \times (\# \text{ access})$$

where $\# \text{ transmit}$ is the count of transmission APIs and $\# \text{ access}$ is the count of access APIs within the slice. This weighting scheme prioritizes hyperedges associated with data exfiltration. The output of this stage is the list H_{API} of slice-aware hyperedges with their calculated weights.

Hypergraph Formation and Representation. Once the three types of hyperedges (H_K , H_{SEM} , H_{API}) are generated, they undergo an Edge Post-processing step (Stage 5). This involves merging all three lists into a single, unified hyperedge set. Duplicate hyperedges are removed, and each hyperedge is capped at a maximum cardinality (e.g., ≤ 64 nodes). This capping is crucial for stabilizing message passing within the hypergraph neural network. Each processed hyperedge is also tagged with a 1-byte type identifier (e.g., ‘topo’, ‘semantic’, ‘suspicious’) and, for H_{API} , its calculated weight w is retained. The output of this stage is a single merged hyperedge set for the APK. Subsequently, the Hypergraph Constructor (Stage 6) takes the vertex set V (methods), the node-feature matrix X (from Stage 3), and the unified hyperedge set (from Stage 5) as input [14]. It then builds the sparse incidence matrix $H \in \{0, 1\}^{|V| \times |E|}$ and creates the diagonal edge-weight matrix W . Weights are typically 1, except for suspicious-API edges where the calculated weight w is used. Symmetric degree normalization is applied to these matrices. Finally, the constructed hypergraph components, $\{H, W, X\}$, are serialized for efficient loading and processing by graph neural network libraries. The output of this stage is one serialized hypergraph object per APK.

Algorithm 1 illustrates the details on Hyperedge generation:

4.4 Mini-batch Builder

For efficient training and inference with hypergraph neural networks, especially when dealing with a large number of individual hypergraphs, a Mini-Batch Builder (Stage 7) is employed. At training or inference time, multiple serialized hypergraphs are processed to create a single batched input. This involves block-diagonalizing [15] their respective incidence matrices (H) and weight matrices (W), and concatenating their feature matrices (X). Additionally, a `batch_vec` is constructed, which maps each node in the concatenated feature matrix back to its original source graph. The output of this stage consists of batched tensors fed into the Hypergraph Attention Neural Network backbone.

Hypergraph Attention Neural Network Backbone. The core of our classification Hypergraph Attention Neural Network backbone (HANN) (Stage 8). Unlike traditional Hypergraph Neural Networks (HGNNs) that use static incidence matrices, our method incorporates an attention mechanism to dynamically

Algorithm 1. Pipeline Stages Group 4**Input:** FCG $G = (V, E)$, embeddings $\mathbf{E}$, slice set $\mathcal{S}$, params $K, k_{nn}, \tau, L_p, c_{\max}$ **Output:** incidence $\mathbf{H}$ (rows,cols), weights $\mathbf{w}$, types $\mathbf{t}$

```

/*Stage 4A: K-hop topological hyperedges
1:  $\mathcal{H} \leftarrow \emptyset$ 
2: for  $v \in V$  do
3:    $e \leftarrow \{u \mid \text{dist}_G(v, u) \leq K\}$ 
4:   if  $2 \leq |e| \leq c_{\max}$  then
5:      $\mathcal{H} \cup = \{(e, 1, \text{topo})\}$ 
6:   end if
7: end for
/*Stage 4B: Semantic (k-NN) hyperedges
8:  $(I, D) \leftarrow \text{FAISS}(\mathbf{E}, k_{nn})$ 
9: for  $i = 0$  to  $|V| - 1$  do
10:   $e \leftarrow \{I_{i,j} \mid D_{i,j} \geq \tau\}$ 
11:  if  $2 \leq |e| \leq c_{\max}$  then
12:     $\mathcal{H} \cup = \{(e, 1, \text{sem})\}$ 
13:  end if
14: end for
/*Stage 4C: Slice-aware suspicious-API hyperedges
15: for  $s \in \mathcal{S}$  do
16:   $e \leftarrow F_s; w \leftarrow 1 + 0.2\#\text{transmit}_s + 0.1\#\text{access}_s$ 
17:  for  $(u, v) \in e \times e$  do
18:    if  $|P| \leq L_p$  where  $P = \text{sp}(G, u, v)$  then
19:       $e \leftarrow e \cup P$ 
20:    end if
21:  end for
22:  if  $2 \leq |e| \leq c_{\max}$  then
23:     $\mathcal{H} \cup = \{(e, w, \text{sus})\}$ 
24:  end if
25: end for

```

weigh the importance of hyperedges during information propagation [6, 20]. This adaptive weighting allows the model to focus on the most critical relationships between functions, leading to enhanced discrimination and noise reduction.

Each layer of the HANN computes node embeddings by aggregating information from connected hyperedges:

$$X' = \sigma(D^{-\frac{1}{2}} H W B^{-1} H^\top D^{-\frac{1}{2}} X P)$$

where X is the input feature matrix, H is the incidence matrix, W is the diagonal weight matrix, D and B are diagonal matrices representing node and hyperedge degrees respectively, σ is an activation function, and P is a learnable projection matrix. For attention layers, the incidence matrix H can be replaced with learned attention weights α . The output of this stage is a set of final node embeddings, h_i , for each function in the hypergraph.

Read-Out and Pooling. Following the HANN backbone, a Read-out/Pooling mechanism (Stage 9) is employed to aggregate the node embeddings (h_i) into a single, fixed-size graph-level embedding, g , for each Android application. This step is crucial for transforming the variable-sized hypergraph representation into a format suitable for downstream classification. We utilize methods such as mean pooling (scatter_mean) or attention pooling (scatter_softmax) to collapse each graph’s nodes to a single vector g . This graph-level embedding encapsulates the overall behavioral characteristics of the application, learned through the nuanced relationships captured by the hypergraph and refined by the attention mechanism. The output of this stage is the per-APK embedding vectors g .

Dual-Head Classification Architecture. The final graph-level embeddings, g , are processed through a sophisticated Dual-Head Classification Architecture (Stage 10) that simultaneously addresses both binary malware detection and fine-grained family classification tasks. The architecture begins with a shared hidden MLP that processes each graph-level embedding g to extract high-level discriminative features. The shared MLP typically consists of two hidden layers with ReLU activation functions and dropout regularization to prevent overfitting [7].

The first component of the dual-head architecture is Head A (Binary Detector), that produces a single logit z_{bin} that is passed through a sigmoid activation function to obtain the malware probability: $p(\text{malware}) = \sigma(z_{\text{bin}})$.

The second component is Head B (Family Classifier), which specializes in identifying specific malware families. This head generates C logits z_{fam} , where C represents the number of distinct malware families in the dataset. These logits are processed through a softmax activation function to produce a probability distribution over the malware families: $p(\text{family}) = \text{softmax}(z_{\text{fam}})$.

The training process employs a **joint loss function** that combines the objectives of both classification heads:

$$\mathcal{L} = \lambda_{\text{bin}} \cdot \text{BCEWithLogits}(z_{\text{bin}}, y_{\text{bin}}) + \lambda_{\text{fam}} \cdot \text{CrossEntropy}(z_{\text{fam}}, y_{\text{fam}}) \quad (5)$$

where λ_{bin} and λ_{fam} are weighting parameters that balance the contribution of each loss component.

The output of this stage provides comprehensive classification results for every APK in the dataset. The integration of hypergraph attention into the Android malware detection framework allows our solution to learn a more detailed representation of the higher-order relationships between functions. By dynamically weighing hyperedges, we achieve enhanced discrimination by emphasizing critical relationships, noise reduction by down-weighting less relevant hyperedges, and adaptive weighting, which provides potential insights into which functional relationships are most critical for malware detection.

5 Evaluation

This section presents a comprehensive evaluation of our proposed system, datasets utilized, evaluation metrics, and a thorough analysis of the results. The

objective is to demonstrate the effectiveness and robustness of our approach in detecting and classifying malware, drawing comparisons with established baseline methods in the field.

5.1 The Dataset and Evaluation Metrics

To rigorously evaluate the performance of our proposed malware detection system, we utilized two widely recognized and publicly available datasets: Drebin [5] and CICMalDroid2020 [26, 27]. These datasets are commonly employed in the Android malware detection research community, allowing for a direct comparison with existing state-of-the-art methods.

Drebin. The Drebin is a prominent dataset in the Android malware research domain, exclusively containing malicious Android applications. Drebin dataset consists exclusively of malicious Android applications, comprising a total of 5,560 samples categorized into 179 distinct malware families. Given the inherent class imbalance within the Drebin dataset, where many malicious families have a small sample size, we focused our evaluation on the top 20 malicious families based on their sample count as detailed in Table 1. This selection, totaling 4,664 malicious samples, ensures that our analysis is conducted on sufficiently represented classes, providing meaningful insights into the model’s ability to distinguish between different malware families. The dataset is split into training, testing, and validation sets with 70% for training, 20% for testing, and 10% for validation.

Table 1. Top 20 malware families in the Drebin dataset

Family	# Sample	Family	# Sample
FakeInstaller	925	Adrd	91
DroidKungFu	667	DroidDream	81
Plankton	625	LinuxLotoor	70
Opfake	613	GoldDream	69
GingerMaster	339	MobileTx	69
BaseBridge	330	FakeRun	61
Iconosys	152	SendPay	59
Kmin	147	Gappusin	58
FakeDoc	132	Imlog	43
Geinimi	92	SMSReg	41
Total		4,664	

CICMalDroid. The CICMalDroid2020 dataset is a comprehensive Android malware collection comprising 17,341 samples gathered from 2017 to 2018 through multiple authoritative sources including VirusTotal, Contagio security blog, AMD, and MalDozer. The dataset encompasses five distinct categories. Adware (1,253 samples), Banking malware (2,100 samples), SMS malware (3,904 samples), Riskware (2,546 samples), and Benign applications (1,795 samples), totaling 11,598 successfully analyzed samples which is detailed in Table 2. By keeping in mind that the CICMalDroid2020 is one of the most feature-rich Android malware datasets available for research community. Similar to Drebin, the CICMalDroid2020 dataset was partitioned into an 70% training, 20% testing, and 10% validation set for experimental purposes.

Table 2. CICMalDroid2020 dataset statistics

Family	# Sample	Distribution (%)
Adware	1,253	10.8
Banking	2,100	18.1
SMS malware	3,904	33.6
Riskware	2,546	22.0
Benign	1,795	15.5
Total	11,598	100

Evaluation Metrics. Due to the imbalanced nature of the datasets, accuracy alone may not reliably reflect classifier performance. Therefore, in this study, both detection and family classification performance are evaluated using additional metrics such as precision, recall, and the Macro-F1. Generally, accuracy is appropriate when true positives and true negatives are equally important and the class distribution is balanced. In contrast, the Macro-F1 is more suitable when false positives and false negatives carry greater significance, especially in scenarios with class imbalance. Since real-world classification tasks involve imbalanced datasets, Macro-F1 provides a more robust measure for model evaluation.

5.2 Evaluation Results

This section presents the experimental results obtained from evaluating our proposed hypergraph-based malware detection system on the Drebin and CICMalDroid2020 datasets. We compare the performance of our method against various baseline approaches, including traditional graph neural networks (GNNs) and other feature-based methods, using the metrics defined in the previous section.

Drebin Performance. For the Drebin dataset, which is characterized by an unbalanced distribution of malicious families, we employed Accuracy, Precision, Recall, and F1-Score to provide a comprehensive evaluation. Table 3 presents the performance of our hypergraph attention-based method (HGANN-Mal) against various graph-based neural network baselines including GCN [21], and GraphSAGE [44] and state-of-the-art hypergraph-based methodologies including, Hypergraph Neural Networks (HGNN) and HGNNP [45]. HGNNP introduces a convolution operator that differs slightly from that of HGNN by utilizing a random-walk-based probability transition matrix for feature propagation [14].

Table 3. Performance of Malware Detection on Derbin

Method	Binary Detection				Multi-Class Classification			
	Macro-F1	Precision	Recall	Accuracy	Macro-F1	Precision	Recall	Accuracy
GCN	92.2	92.2	91.5	92.8	86.1	85.9	86.2	87.8
GraphSAGE	92.1	92.1	92.5	93.3	86.3	86.7	86.2	88.0
HGNN	96.2	95.8	96.3	96.8	92.0	92.1	92.4	92.0
HGNNP	96.1	95.9	96.1	97.1	92.3	92.5	92.5	93.3
HGANN-Mal	97.8	97.8	97.6	98.3	94.2	94.1	93.8	95.0

Table 3 indicates that HGANN-Mal, our attention hypergraph-based method consistently achieved high performance across all metrics on the Drebin dataset. HGNNP attained the optimal accuracy of 97.1%, while HGANN-Mal demonstrated the best Precision (97.8%), and Macro-F1 (97.8%) for binary malware detection. Compared to GCN and GraphSAGE (the graph-based baselines), HGNN, HGNNP, and HGANN-Mal showed significant improvements in accuracy (min 3.5% and max 5.5%) and Macro-F1 (min 3.9% and max 5.7%), respectively for binary detection. Even when compared to GraphSAGE, which is known for its generalization capabilities due to neighbor sampling, the hypergraph-based still yielded superior results in terms of accuracy and Macro-F1, with improvements. This highlights the substantial benefits of integrating hypergraphs into Android malware detection, enabling more accurate and robust classification of complex and evolving threats.

However, regarding the Multi-class classification task which tries to categorize the malware families, again, HGANN-Mal outperformed other baseline and state-of-the-art models by achieving the Macro-F1 score of 94.2%. It is worth mentioning that such high scores are achieved only through a static analysis approach, which is less expensive than dynamic feature extraction. This further supports the notion that hypergraph modeling effectively extracts richer higher-order information, which is crucial for classifying malicious applications, for instance, in datasets with diverse and unbalanced malware families. Unlike static hypergraph models, which treat all neighbor nodes or hyperedges as equally important, hypergraph attention networks introduce a trainable attention mech-

anism to dynamically learn and assign weights during message passing resulting in a higher performance numbers.

CICMalDroid Performance. For the CICMalDroid2020 dataset, HGANN-Mal demonstrates exceptional performance across both binary detection and multi-class classification tasks. In binary detection, the model achieves outstanding results with a Macro-F1 score of 96.2%, precision of 96.2%, recall of 97.6%, and accuracy of 97.6%. These metrics significantly outperform baseline methods, with HGNNP achieving 95.9% Macro-F1 and traditional approaches like GCN and GraphSAGE scoring 90.6% and 91.0% respectively.

Notably, while HGANN-Mal excels in most metrics, HGNNP achieves a slightly higher accuracy of 97.7% compared to HGANN-Mal’s 97.6%. This marginal accuracy difference suggests that the attention mechanism in HGANN-Mal may introduce a subtle trade-off between precision-recall balance and raw accuracy optimization (Table 4).

Table 4. Performance of Malware Detection on CICMalDroid2020

Method	Binary Detection				Multi-Class Classification			
	Macro-F1	Precision	Recall	Accuracy	Macro-F1	Precision	Recall	Accuracy
GCN [21]	90.6	90.6	90.6	90.9	82.4	82.0	82.4	82.7
GraphSAGE [44]	91.0	91.1	91.1	91.5	82.3	82.2	82.6	83.4
HGNN [45]	94.2	94.8	94.3	94.6	88.7	88.8	88.8	89.0
HGNNP [45]	95.9	96.1	96.5	97.7	89.3	89.5	89.8	90.1
HGANN-Mal	96.2	96.2	97.6	97.6	94.0	94.1	94.0	94.2

The multi-class classification results further validate the model’s effectiveness, with HGANN-Mal achieving a Macro-F1 score of 94.0%, precision of 94.1%, recall of 94.0%, and accuracy of 94.2%. Compared to HGNNP’s 89.3% Macro-F1 and other baseline methods scoring below 90%, HGANN-Mal demonstrates superior capability in distinguishing between different malware families within the CICMalDroid dataset. These results highlight the model’s robust performance on the CICMalDroid2020 dataset, confirming its effectiveness in handling diverse and complex malware detection scenarios with high accuracy and reliability.

6 Limitations and Future Work

Our methodology, integrating hypergraph attention into the Android malware detection framework, is designed to overcome the limitations of traditional graph-based and static hypergraph-based approaches by learning more comprehensive representations of higher-order relationships between functions. Traditional FCGs often simplify these interactions into binary edges, losing crucial contextual information.

The consistent outperformance of HGANN-Mal across both the Drebin and the CICMalDroid2020 datasets underscores the robustness and generalizability of our proposed approach, which are critical for real-world malware detection systems. Since Drebin is a widely recognized benchmark in Android malware detection, enabling direct comparison with existing state-of-the-art methods; however, roughly 49.35% of its samples are repackaged duplicates sharing identical opcode sequences, risking train-test leakage and inflated performance estimates. To mitigate single dataset bias and better reflect recent Android malware trends, we also evaluate on CICMalDroid2020 dataset.

While the results are promising, it is important to acknowledge certain considerations. Our evaluation focuses on HGNN/GCN baselines aligned with our static pipeline and does not include the Android detectors such as MaMaDroid [32] (and Drebin), which limits direct comparability to the broader literature. We will incorporate these non-hypergraph baselines in future work.

Also, the construction and processing of hypergraphs can be computationally more intensive than traditional graphs, especially for very large and dense hypergraphs. Hypergraphs are designed to capture higher-order relationships, sparse data (e.g., applications with very few functions or limited API calls) might lead to less informative hyperedges or difficulties in learning robust attention weights. Future work could explore more efficient algorithms for hypergraph construction and optimization techniques for training HNNs on massive datasets.

7 Conclusion

In this paper, we introduced HGANN-Mal, a novel framework for Android malware detection that leverages an attention-based hypergraph neural network to overcome the limitations of conventional graph-based methods. Our approach constructs a hypergraph from static analysis features to model complex, higher-order relationships within applications. The core innovation is an attention mechanism that dynamically weighs these relationships based on their learned relevance to malicious behavior, enabling our model to adaptively focus on the most suspicious patterns and enhance classification accuracy. Experimental results on the Drebin and CICMalDroid datasets confirm that HGANN-Mal significantly outperforms state-of-the-art methods in both binary and multi-class classification tasks. This work demonstrates that an attention-based hypergraph approach provides a more nuanced, robust, and powerful tool for the complex task of malware detection. Future work will explore the integration of dynamic analysis features and investigate the interpretability of the learned attention weights to provide actionable security insights.

References

1. Agarwal, S., Branson, K., Belongie, S.: Higher order learning with graphs. In: Proceedings of the 23rd International Conference on Machine Learning (2006)

2. Alon, U., et al.: Code2vec: learning distributed representations of code. *Proc. ACM Program. Lang.* **3**, 1–29 (2019)
3. Amer, E.: A dynamic windows malware detection and prediction method based on contextual understanding of api call sequence. *Comput. Secur.* (2020)
4. Arora, A.A.E.A.: Permpair: android malware detection using permission pairs. *IEEE Trans. Inf. Forensics Secur.* (2020)
5. Arp, D., et al.: Drebin: effective and explainable detection of android malware in your pocket. In: *NDSS* (2014)
6. Bai, S., Zhang, F., Torr, P.H.S.: Hypergraph convolution and hypergraph attention. *Pattern Recogn.* **110** (2021)
7. Banerjee, C., et al.: An empirical study on generalizations of the relu activation function. In: *Proceedings of the 2019 ACM Southeast Conference* (2019)
8. Bilot, T., El Madhoun, N., Al Agha, K., Zouaoui, A.: A survey on malware detection with graph representation learning. *ACM Comput. Surv.* **56**(11) (2024)
9. Clevert, D.A., Unterthiner, T., Hochreiter, S.: Fast and accurate deep network learning by exponential linear units (elus). *arXiv preprint [arXiv:1511.07289](https://arxiv.org/abs/1511.07289)* (2015)
10. Cui, H., et al.: On positional and structural node features for graph neural networks on non-attributed graphs. In: *31st ACM International Conference on Information & Knowledge Management* (2022)
11. Feng, Y., You, H., Zhang, Z., Ji, R., Gao, Y.: Hypergraph neural networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence* (2019)
12. Feng, Y., et al.: Apposcopy: semantics-based detection of android malware through static analysis. In: *22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (2014)
13. Freeman, L.C.: A set of measures of centrality based on betweenness. *Sociometry* **35**–41 (1977)
14. Gao, Y., et al.: Hgcn+: general hypergraph neural networks. *IEEE Trans. Pattern Anal. Mach. Intell.* (2023)
15. Gao, Y., Zhang, Z., Lin, H., Zhao, X., Du, S., Zou, C.: Hypergraph learning: methods and practices. *IEEE Trans. Pattern Anal. Mach. Intell.* **44**(5), 2548–2566 (2020)
16. Gascon, H., et al.: Structural detection of android malware using embedded call graphs. In: *2013 ACM Workshop on Artificial Intelligence and Security* (2013)
17. Google: The android show: I/o edition (May 2025). <https://blog.google/products/android/the-android-show-io-2025/>. Accessed 11 June 2025
18. Guo, W., et al.: Malhapgcn: an enhanced call graph-based malware detection framework using hierarchical attention pooling graph neural network. *Sensors* **25**(2), 374 (2025)
19. Kang, B., Yerima, S.Y., Sezer, S., McLaughlin, K.: N-gram opcode analysis for android malware detection. *arXiv preprint [arXiv:1612.01445](https://arxiv.org/abs/1612.01445)* (2016)
20. Khan, B., Wu, J., Yang, J., Ma, X.: Heterogeneous hypergraph neural network for social recommendation using attention network. *ACM Trans. Recomm. Syst.* **3**(3), 1–22 (2025)
21. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. *arXiv preprint [arXiv:1609.02907](https://arxiv.org/abs/1609.02907)* (2016)
22. Lee, J.B., Rossi, R.A., Kim, S., Ahmed, N.K., Koh, E.: Attention models in graphs: a survey. *ACM Trans. Knowl. Discov. Data (TKDD)* **13**(6), 1–25 (2019)
23. Li, C., et al.: Dmalnet: dynamic malware analysis based on api feature engineering and graph learning. *Comput. Secur.* **122**, 102872 (2022)
24. Lu, X.A.E.A.: Sndgcn: robust android malware detection based on subgraph network and denoising gcnn network. *Expert Syst. Appl.* (2024)

25. Maas, A.L., Hannun, A.Y., Ng, A.Y., et al.: Rectifier nonlinearities improve neural network acoustic models. In: Proceedings of the ICML, vol. 30, p. 3. Atlanta, GA (2013)
26. MahdaviFar, S., et al.: Dynamic android malware category classification using semi-supervised deep learning. In: The 18th IEEE International Conference on Dependable, Autonomic, and Secure Computing (DASC) (2020)
27. MahdaviFar, S., et al.: Effective and efficient hybrid android malware classification using pseudo-label stacked auto-encoder. *J. Netw. Syst. Manag.* (2022)
28. Moser, A., Kruegel, C., Kirda, E.: Limits of static analysis for malware detection. In: Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC) (2007)
29. Narayanan, A., Chandramohan, M., Chen, L., Liu, Y.: A multi-view context-aware approach to android malware detection and malicious code localization. *Empir. Softw. Eng.* **23**(3) (2018)
30. Niu, W., Cao, R., Zhang, X., Ding, K., Zhang, K., Li, T.: Opcode-level function call graph based android malware classification using deep learning. *Sensors* **20**(13), 3645 (2020)
31. Norouzian, M.R., Xu, P., Eckert, C., Zarras, A.: Hybrid: toward android malware detection and categorization with program code and network traffic. In: Liu, J.K., Katsikas, S., Meng, W., Susilo, W., Intan, R. (eds.) *ISC 2021. LNCS*, vol. 13118, pp. 259–278. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-91356-4_14
32. Onwuzurike, L., et al.: Mamadroid: detecting android malware by building markov chains of behavioral models (extended version). *ACM Trans. Priv. Secur. (TOPS)* (2019)
33. Payet, É., Spoto, F.: Static analysis of android programs. *Inf. Softw. Technol.* **54**(11), 1192–1201 (2012)
34. Qian, X., Zheng, X., He, Y., Yang, S., Cavallaro, L.: Lamd: context-driven android malware detection and classification with llms. *arXiv preprint arXiv:2502.13055* (2025)
35. Qiu, J., et al.: Cyber code intelligence for android malware detection. *IEEE Trans. Cybern.* **53**(1), 617–627 (2022)
36. Shokouhinejad, H., et al.: Recent advances in malware detection: graph learning and explainability. *arXiv preprint arXiv:2502.10556* (2025)
37. Soi, D., Sanna, A., Maiorca, D., Giacinto, G.: Enhancing android malware detection explainability through function call graph apis. *J. Inf. Secur. Appl.* **80**, 103691 (2024)
38. Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based graph neural networks: a survey. *Artif. Intell. Rev.* **56**(Suppl 2), 2263–2310 (2023)
39. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Li, P., Bengio, Y.: Graph attention networks. In: International Conference on Learning Representations (ICLR) (2018)
40. Wu, D.J., et al.: Droidmat: android malware detection through manifest and api calls tracing. In: Seventh Asia Joint Conference on Information Security (2012)
41. Wu, L., Wang, D., Song, K., Feng, S., Zhang, Y., Yu, G.: Dual-view hypergraph neural networks for attributed graph learning. *Knowl.-Based Syst.* **227**, 107185 (2021)
42. Xiao, X., Xiao, X., Jiang, Y., Liu, X., Ye, R.: Identifying android malware with system call co-occurrence matrices. *Trans. Emerg. Telecommun. Technol.* **27**(5), 675–684 (2016)
43. You, I., Yim, K.: Malware obfuscation techniques: a brief survey. In: BWCCA (2010)

44. Yumlembam, R., Issac, B., Jacob, S.M., Yang, L.: Iot-based android malware detection using graph neural network with adversarial defense. *IEEE Internet Things J.* **10**(10), 8432–8444 (2022)
45. Zhang, D., et al.: Android malware detection based on hypergraph neural networks. *Appl. Sci.* **13**(23) (2023)
46. Zheng, J., et al.: Maskdroid: robust android malware detection with masked graph representations. In: *39th IEEE/ACM ASE 2024* (2024)
47. Zhou, D., Huang, J., Schölkopf, B.: Learning with hypergraphs: clustering, classification, and embedding. *Adv. Neural Inf. Process. Syst.* (2006)